

SmartTask – جزوه جامع پروژه کارشناسی

نام پروژه: SmartTask – سامانه مدیریت هوشمند وظایف چابک نوع پروژه: پایان نامه کارشناسی تکنولوژی اصلی: ASP.NET Core MVC 8.0 پایگاه داده: Microsoft SQL Server زبان هوش مصنوعی: مدل Qwen3-4B (از طریق LM Studio روی سرور لوکال) fpF: ساختار Agile / Scrum

فهرست مطالب

- نمای کلی پروژه
- معماری و زیرساخت فنی
- لایه بندی معماری (Architecture Layers)
- موجودیت های داده ای (Database Entities)
- فیچرهای پایه ای سامانه
- نواوری های اصلی پروژه (۱۱ نواوری)
- تکنولوژی های استفاده شده و چرایی استفاده از هر کدام
- فایل ساختار پروژه
- تست ها و کیفیت کد
- نقشه راه و وضعیت پیاده سازی

۱. نمای کلی پروژه

SmartTask چیست؟

SmartTask یک سامانه وب محور مدیریت وظایف چابک (Agile Task Management) است که بر اساس روش شناسی Scrum طراحی شده. هدف اصلی پروژه این است که فراتر از ابزارهای ساده مدیریت تسک، ابزارهای هوشمند و خودکاری در اختیار مدیران پروژه و اعضای تیم قرار دهد تا:

- بار کاری اعضا به صورت خودکار تحلیل شود
- وابستگی بین وظایف شناسایی و تأثیر تأخیرها به صورت زنجیره ای محاسبه شود
- اولویت وظایف بر اساس الگوریتم هوشمند (نه فقط حس مدیر) تعیین شود
- ریسک تأخیر پروژه پیش بینی و به صورت بصری نمایش داده شود
- گزارش های اسپرینت توسط هوش مصنوعی تولید شود
- تجزیه وظایف توسط هوش مصنوعی و به صورت خودکار انجام شود
- مبادله تسک بین اعضا با سیستم درخواست و تأییدیه مدیریت شود

- ** کارهای فوری خارج از دامنه** (آفرود) بدون آسیب به ساختار رسمی پروژه ثبت شوند
- یک معماری تصمیم‌یار مبتنی بر LLM داده‌های ساختاریافته پروژه را تحلیل کرده و پیشنهادهای مدیریتی ارائه دهد

مخاطبان سامانه

نقش	دسترسی
Admin	مدیریت کل سامانه، کاربران، داشبورد ادمین
ProjectManager	مدیریت پروژه‌ها، اسپرینت‌ها، تیم‌ها، مشاهده تحلیل‌ها
Member	ثبت وظایف، ترید تسک، مشاهده داشبورد

ساختار سازمانی سامانه

```

Workspace (فضای کاری)
├── Project (پروژه)
│   ├── Backlog (بکلاگ)
│   │   ├── UserStory (داستان کاربری)
│   │   │   ├── TaskItem (وظیفه اجرایی)
│   │   │   │   ├── SubTaskItem (زیروظیفه)
│   │   │   │   ├── TaskAssignment (تخصیص به عضو)
│   │   │   │   ├── TaskDependency (وابستگی)
│   │   │   │   ├── Comment (نظر)
│   │   │   │   ├── Attachment (پیوست)
│   │   │   │   ├── Checklist (چکلیست)
│   │   │   │   ├── Label (برچسب)
│   │   │   │   └── TimeLog (ثبت زمان)
│   │   └── Sprint (اسپرینت)
│   │       └── UserStory → TaskItem
│   ├── OffroadTask (کارهای آفرود)
│   ├── TaskTradeRequest (درخواست ترید)
│   └── ProjectMember (اعضای پروژه)
└── Team (تیم)
    └── TeamMember
  
```

۲. معماری و زیرساخت فنی

تکنولوژی‌های استفاده‌شده

لایه	تکنولوژی	نسخه
Framework	ASP.NET Core	8.0
ORM	Entity Framework Core	8.0.28
Authentication	ASP.NET Core Identity	8.0.8

نسخه	تکنولوژی	لایه
(built-in)	SignalR	Real-time
-	Microsoft SQL Server	Database
2026.7.2	QuestPDF	PDF
0.105.1	ClosedXML	Excel
-	OpenAI-compatible API (LM Studio + Qwen3-4B)	AI
-	System.Net.Mail (SMTP)	Email
API v1	Webpushr	Push Notification
-	xUnit + Moq + EF InMemory	Testing

چرا ASP.NET Core 8.0؟

- عملکرد بالا: Kestrel web server بهینه شده
- **Dependency Injection** داخلی و قدرتمند
- **SignalR** برای ارتباط بلادرنگ (Real-time)
- **Entity Framework Core** برای ORM قدرتمند
- پشتیبانی عالی از **Identity** برای احراز هویت و نقش‌ها

چرا SQL Server؟

- پشتیبانی از توابع پیچیده‌ی Query
- مقیاس‌پذیری بالا
- سازگاری کامل با Entity Framework Core

چرا SignalR؟

SignalR چیست؟ SignalR یک کتابخانه میکروسافت برای اضافه کردن قابلیت ارتباط بلادرنگ (Real-time) به اپلیکیشن‌های وب است. به زبان ساده، به جای اینکه مرورگر مدام صفحه را رفرش کند تا پیام جدیدی بیاید، سرور خودش پیام را فوراً به مرورگر می‌فرستد.

چرا در SmartTask استفاده شد؟

1. **اعلان‌های فوری:** وقتی Task جدیدی تخصیص داده شود، اعلان فوراً به کاربر می‌رسد
2. **پیام‌رسانی گروهی چت:** چت پروژه بدون رفرش صفحه کار می‌کند
3. **وضعیت آنلاین/آفلاین:** نشان دادن اینکه کی آنلاین است
4. **تایپ کردن:** نشان دادن «...در حال تایپ» به سایرین

کجا در پروژه استفاده شد؟

- فایل Hubs/NotificationHub.cs — هاب اعلان‌ها
- فایل Hubs/ChatHub.cs — هاب چت گروهی پروژه‌ها

- فایل Program.cs — ثبت هاب‌ها: `app.MapHub<NotificationHub>("/hubs/notification")` و `app.MapHub<ChatHub>("/hubs/chat")`
- فایل Services/Implementations/NotificationService.cs — ارسال اعلان از سمت سرور با `hubContext.Clients.Group(...).SendAsync("ReceiveNotification", ...)`

۳. لایه‌بندی معماری

SmartTask از معماری **Clean Architecture** ساده‌شده استفاده می‌کند:

Controllers (MVC)	← لایه ارائه (Presentation)
Services (Implementations)	← لایه منطق تجاری (Business Logic)
AI Decision Support Layer	← (الگوریتم + LLM) لایه تصمیم‌یار
Infrastructure	← Repository + UnitOfWork + BackgroundJobs
Data (DbContext + Config)	← لایه دسترسی به داده (Data Access)
Models (Entities + Enums)	← مدل‌های داده‌ای

۳.۱ لایه Models

محل قرارگیری: `/SmartTask.Web/Models`

پوشه	محتوا
<code>/Entities</code>	کلاس‌های موجودیت داده‌ای (34 فایل)
<code>/Enums</code>	شمارشگرهای وضعیت و اولویت (23 فایل)
<code>/ViewModels</code>	مدل‌های نمایشی برای View ها (30+ پوشه)
<code>/DTOs</code>	مدل‌های انتقال داده (BurndownPointDto, VelocityPointDto)

BaseEntity: تمام موجودیت‌ها از یک کلاس پایه به ارث می‌برند:

```
// SmartTask.Web/Models/Entities/BaseEntity.cs
public abstract class BaseEntity {
    public int Id { get; set; }
    public string? CreatedBy { get; set; }
    public DateTime CreatedDate { get; set; }
    public string? ChangeUser { get; set; }
    public DateTime? ChangeDate { get; set; }
    public bool ViewState { get; set; } = true; // Soft Delete
}
```

نکته مهم: فیلد `ViewState` برای **Soft Delete** استفاده می‌شود. یعنی رکوردها هرگز حذف فیزیکی نمی‌شوند و فقط `ViewState = false` می‌شوند. این کار باعث حفظ تاریخچه و قابلیت بازیابی می‌شود.

۳.۲ لایه Services

محل قرارگیری: `/SmartTask.Web/Services`

هر سرویس شامل دو بخش است:

- `/Interface: Services/Interfaces` – تعریف قرارداد (45 فایل)
- `/Implementation: Services/Implementations` – پیاده‌سازی (45 فایل)

چرا Interface و Implementation جدا هستند؟

- **تست‌پذیری:** می‌توان در تست‌ها Mock جایگزین کرد
- **تغییر پیاده‌سازی:** مثلاً می‌توان سرویس ایمیل را بدون تغییر کل کد عوض کرد
- **Dependency Injection:** فریمورک خودش تشخیص می‌دهد کدام پیاده‌سازی را تزریق کند

۳.۳ لایه Infrastructure

محل قرارگیری: `/SmartTask.Web/Infrastructure`

پوشه	محتوا	توضیح
<code>/Repositories</code>	<code>GenericRepository<T></code> + <code>UnitOfWork</code>	الگوی Repository و Unit of Work
<code>/BackgroundJobs</code>	<code>OverdueCascadeBackgroundService</code> + <code>ReminderBackgroundService</code>	سرویس‌های پس‌زمینه
<code>/Seed</code>	<code>RoleSeeder</code> + <code>AdminSeeder</code>	ایجاد نقش‌ها و ادمین اولیه
<code>/Services</code>	<code>CurrentUserService</code> + <code>CurrentContextService</code> + <code>PresenceTracker</code>	سرویس‌های کمکی

الگوی Generic Repository:

```
// Infrastructure/Repositories/GenericRepository.cs
public class GenericRepository<T> : IGenericRepository<T> where T : class {
    protected readonly ApplicationDbContext _context;
    protected readonly DbSet<T> _dbSet;
    // GetAllAsync, GetByIdAsync, FindAsync, AddAsync, Update, Delete, SaveChangesAsync
}
```

چرا Generic Repository؟

- جلوگیری از تکرار کد دسترسی به دیتابیس
- هر موجودیت فقط یک‌بار عملیات CRUD تعریف می‌کند
- واحد کار (`UnitOfWork`) تضمین می‌کند چند تغییر در یک تراکنش ذخیره شوند

۳.۴ لایه Controllers

محل قرارگیری: 41 – SmartTask.Web/Controllers/ کنترلر

مهم‌ترین کنترلرها:

وظیفه	کنترلر
CRUD پروژه‌ها	ProjectController
مدیریت اسپرینت‌ها	SprintController
مدیریت وظایف	TaskController
مدیریت وابستگی‌ها	TaskDependencyController
کارهای آفرود	OffroadController
تحلیل بارکاری	WorkloadController
پیش‌بینی ریسک	DelayRiskController
ترید تسک	TaskTradeController
گزارش اسپرینت	SprintReportController
چت گروهی	ChatController
اعلان‌ها	NotificationController

۳.۵ لایه Data

محل قرارگیری: SmartTask.Web/Data/

- Context/ApplicationDbContext.cs – کلاس اصلی Context دیتابیس
- Configurations/ – 35 – فایل پیکربندی EF Core برای هر موجودیت

۳.۶ لایه تصمیم‌یار هوش مصنوعی (AI Decision Support Layer)

SmartTask یک لایه تصمیم‌یار مبتنی بر LLM دارد که چرخه کامل زیر را پوشش می‌دهد:

شاخص عددی → پیشنهاد → پذیرش/رد مدیر → ثبت تصمیم → LLM → داده دیتابیس → تحلیل ساختاریافته

فایل‌های کلیدی:

فایل	وظیفه
Services/AI/IAIClientService.cs	رابط AI با متد <GetStructuredCompletionAsync>T
Services/AI/AiClientService.cs	پیاده‌سازی – ارسال Prompt و Deserialize خروجی JSON

فایل	وظیفه
/Models/ViewModels/Ai	مدل‌های خروجی AI (3 فایل)
Models/Entities/AiDecisionLog.cs	لاگ تصمیمات مدیر
Models/Enums/AiDecisionType.cs	نوع تصمیم (Priority, Risk, Health, ...)
Models/Enums/AiUserDecision.cs	تصمیم مدیر (Accepted, Rejected, Ignored)
Services/Implementations/AiDecisionLogService.cs	ثبت تصمیمات AI

چرخه تصمیم‌یار:

1. استخراج داده: سرویس‌های الگوریتمی (Workload, Priority, Risk, Health) از دیتابیس داده جمع‌آوری می‌کنند
2. تحلیل الگوریتمی: امتیاز عددی محاسبه می‌شود (مثلاً امتیاز ریسک 0-100)
3. ارسال به LLM: داده‌های ساختاریافته به صورت Prompt به هوش مصنوعی ارسال می‌شود
4. دریافت خروجی LLM: خروجی ساختاریافته برمی‌گرداند (نه متن آزاد)
5. Parse و ترکیب: خروجی LLM با تحلیل الگوریتمی ترکیب می‌شود
6. پیشنهاد: مدیر پیشنهاد ترکیبی (الگوریتم + AI) را مشاهده می‌کند
7. پذیرش/رد: مدیر تصمیم می‌گیرد و در سیستم ثبت می‌شود

مزیت نسبت به «فقط اتصال AI»:

سیستم فقط متن نمی‌نویسد – خروجی AI به شاخص‌های عددی پروژه تبدیل می‌شود و مدیر می‌تواند تصمیم بگیرد. تصمیمات مدیر ثبت می‌شود تا صحت AI قابل ارزیابی باشد.

۴. موجودیت‌های داده‌ای

جدول موجودیت‌های اصلی

ApplicationUser (کاربر)

فایل: Models/Entities/ApplicationUser.cs

- از IdentityUser<int> ارث‌بری می‌کند (احراز هویت)
- فیلدها: FirstName, LastName, FullName (محاسباتی), Avatar, Bio, JobTitle
- تنظیمات شخصی: Theme (روشن/تاریک), DateFormat, TimeZone (میلادی/جلالی)
- AutoCascadeDependencyDates – آیا تمدید خودکار وابستگی فعال باشد
- WebpushSubscriberId – شناسه اشتراک Push Notification
- WeeklyCapacityHours در ProjectMember – ظرفیت کاری هفتگی

Project (پروژه)

فایل: Models/Entities/Project.cs

- فیلدها: Name, Key, Description, Color, Icon, StartDate, DueDate, EndDate
- Status: Planning, Active, OnHold, Completed, Cancelled
- Priority: Low, Medium, High, Critical
- ارتباط با: Workspace, Members, Sprints, UserStories, Backlog, Labels, ProjectTeams

Sprint (اسپرینت)

فایل: Models/Entities/Sprint.cs

- فیلدها: Name, Goal, StartDate, EndDate, Capacity
- Status: Planning, Active, Review, Completed

UserStory (داستان کاربری)

فایل: Models/Entities/UserStory.cs

- فیلدها: Title, Description, AcceptanceCriteria, StoryPoint, BusinessValue, Order
- Priority: Lowest, Low, Medium, High, Highest
- Status: New, InProgress, Done, Cancelled
- ارتباط با: Project, Backlog, Sprint, Owner, Tasks

TaskItem (وظیفه اجرایی)

فایل: Models/Entities/TaskItem.cs

- فیلدها: StartDate, DueDate, CompletedDate (ساعت), Title, Description, Status, Priority, Type, Estimate
- Status: ToDo, InProgress, InReview, Done, Cancelled
- Priority: Lowest, Low, Medium, High, Highest
- Type: Task, Bug, Feature, Improvement, TechnicalDebt
- ارتباطات: UserStory, SubTasks, Assignments, Comments, Attachments, Checklists, TaskLabels, Reminders, ActivityLogs, TimeLogs

TaskDependency (وابستگی)

فایل: Models/Entities/TaskDependency.cs

- فیلدها: TaskItemId, DependsOnTaskItemId, IsRequired
- جدول واسط بین دو TaskItem

OffroadTask (کار آفرود)

فایل: Models/Entities/OffroadTask.cs

- فیلدها: ProjectId, Title, Description, Status, Priority, CreatedByUserId, AssignedToUserId, DueDate

- Status: ToDo, InProgress, Done, Cancelled

- Priority: Low, Normal, High, Critical

TaskTradeRequest (درخواست ترید)

فایل: `Models/Entities/TaskTradeRequest.cs`

- فیلدها: ProjectId, RequesterUserId, TargetUserId, RequesterTaskId, TargetTaskId, Message, Status, ResponseDate

- Status: Pending, Accepted, Rejected, Cancelled

OverdueCascadeLog (لاگ تمدید خودکار)

فایل: `Models/Entities/OverdueCascadeLog.cs`

- فیلدها: SourceTaskId, ImpactedTaskId, DelayDaysApplied, AppliedDate

- برای جلوگیری از اعمال تکراری تمدید

SprintReport (گزارش اسپرینت)

فایل: `Models/Entities/SprintReport.cs`

- فیلدها: SprintId, Content, GeneratedByUserId, GeneratedDate

- ذخیره گزارش‌های تولیدشده توسط AI

Notification (اعلان)

فایل: `Models/Entities/Notification.cs`

- فیلدها: ApplicationUserId, Title, Message, Type, IsRead, ReadDate

- Type: System, Reminder, Assignment, Comment, Mention, Invitation, StatusChange, Deadline

۵. فیچرهای پایه‌ای سامانه

۵.۱ احراز هویت و مدیریت نقش‌ها

- فناوری: ASP.NET Core Identity

- فایل‌ها: `Controllers/AccountController.cs` , `Infrastructure/Seed/RoleSeeder.cs` , `Infrastructure/Seed/AdminSeeder.cs`

`Infrastructure/Seed/AdminSeeder.cs`

- نقش‌ها: Admin, ProjectManager, Member

- سیاست‌های دسترسی: AdminOnly, ProjectManagerOnly, MemberOnly

- تنظیمات کوکی: نام کوکی `SmartTask` , انقضا ۷ روز , Sliding Expiration فعال

۵.۲ Workspace (فضای کاری)

- ایجاد و مدیریت فضاهای کاری
- دعوت اعضا از طریق ایمیل
- مدیریت نقش اعضا در Workspace

۵.۳ مدیریت پروژه

- CRUD پروژه‌ها با رنگ و آیکون سفارشی
- تخصیص تیم و اعضای پروژه
- مدیریت برچسب‌ها (Label)

۵.۴ مدیریت User Stories و Backlog

- Drag & Drop با قابلیت Product Backlog
- Business Value و Story Point با User Stories
- انتقال User Story به Sprint

۵.۵ مدیریت Sprint

- ایجاد، شروع و پایان اسپرینت
- Sprint Board (تخته اسپرینت) با ستون‌های ToDo, InProgress, InReview, Done

۵.۶ مدیریت Task

- CRUD وظایف با جزئیات کامل
- تخصیص چند عضو به یک Task
- ثبت زمان (Time Log)
- چک لیست داخلی
- پیوست فایل
- نظرات و بحث
- برچسب‌گذاری

۵.۷ سیستم اعلان (Notification)

- فایل‌ها: `Hubs/NotificationHub.cs` , `Services/Implementations/NotificationService.cs`
- اعلان‌های بلادرنگ با SignalR
- انواع اعلان: System, Reminder, Assignment, Comment, Mention, Invitation, StatusChange, Deadline
- تنظیمات شخصی اعلان‌ها
- خواندن/حذف تکی و گروهی

۵.۸ چت گروهی پروژه

- فایل‌ها: `Hubs/ChatHub.cs` , `Services/Implementations/ChatService.cs`

- پیام‌رسانی بلادرنگ با SignalR
- ارسال فایل و تصویر
- پاسخ به پیام (Reply)
- ویرایش و حذف پیام
- واکنش‌های ایموجی (Reaction)
- سنجاق کردن پیام (Pin)
- ذکر کردن (Mention@)
- وضعیت تایپ (Typing Indicator)
- وضعیت آنلاین/آفلاین اعضا
- Rate Limiting (محدودیت ارسال: ۵ پیام در ۳ ثانیه)
- Webpushr با Push Notification

۵.۹ یادآوری (Reminder)

- فایل: `Services/Implementations/ReminderService.cs`
- یادآوری دستی توسط کاربر
- یادآوری خودکار (Auto Reminder)
- سرویس پس‌زمینه `ReminderBackgroundService` که دوره‌ای یادآوری‌های ارسال‌نشده را بررسی می‌کند

۵.۱۰ داشبوردها

- **داشبورد ادمین:** `AdminDashboardService` — آمار کلی کاربران و پروژه‌ها
- **داشبورد کاربر:** `UserDashboardService` — تسک‌های من، اعلان‌ها، فعالیت‌ها
- **داشبورد پروژه:** `ProjectDashboardService` — آمار اسپرینت، نمودار Burndown/Velocity
- **داشبورد Workspace:** `WorkspaceDashboardService`

۵.۱۱ گزارش‌گیری

- خروجی PDF با QuestPDF
- خروجی Excel با ClosedXML
- نمودار Velocity و Burndown

۶. نوآوری‌های اصلی پروژه

نکته کلیدی برای دفاع: این ۱۱ نوآوری دقیقاً همان بخش‌هایی هستند که SmartTask را از ابزارهای مدیریت تسک ساده متمایز می‌کنند. هر کدام مشکل مشخصی را حل می‌کنند.

نواوری ۱: تجزیه هوشمند وظایف با هوش مصنوعی (AI Task Breakdown)

مشکلی که حل می‌کند: وقتی یک تسک پیچیده ثبت می‌شود، اعضای تیم (به خصوص افراد کم‌تجربه) نمی‌دانند از کجا شروع کنند. باید خودشان تسک را به زیروظایف کوچک‌تر تقسیم کنند که کاری زمان‌بر و نیازمند تجربه است.

راه حل SmartTask: با یک کلیک، هوش مصنوعی تسک را به ۳ تا ۷ زیروظیفه عملی تقسیم می‌کند.

پیاده‌سازی فنی:

جزئیات	مقدار
سرویس اصلی	TaskBreakdownService.cs
رابط	ITaskBreakdownService.cs
کنترلر	SubTaskController.cs
سرویس AI	AIClientService.cs
مدل AI	Qwen3-4B
آدرس API	http://92.246.145.99:1234/v1/chat/completions
پرامپت سیستم	فقط JSON array از رشته فارسی (۲ تا ۷ آیتم، هرکدام حداکثر ۸ کلمه)
Temperature	0.5 (برای پاسخ‌های دقیق‌تر)
Max Tokens	400

نحوه عملکرد:

- کاربر تسک را مشاهده می‌کند و دکمه «تجزیه با هوش مصنوعی» را می‌زند
- `TaskBreakdownService.GenerateSubTasksAsync()` فراخوانی می‌شود
- عنوان، توضیحات، نوع و اولویت تسک به صورت Prompt به AI ارسال می‌شود
- AI یک JSON array برمی‌گرداند
- پاسخ پارس می‌شود (اگر JSON نبود، به صورت خطی تقسیم می‌شود)
- لیست زیروظایف به کاربر نمایش داده می‌شود
- کاربر می‌تواند آن‌ها را انتخاب و ذخیره کند

تکنیک‌های هوشمند:

- پارس کردن هم JSON و هم متن ساده (Fallback)
- حداکثر ۷ آیتم و حداکثر ۸ کلمه برای هر آیتم
- فیلتر کردن خطوط خالی

نواوری ۲: بخش آفرود (Offroad Tasks)

مشکلی که حل می‌کند: در هر پروژه، کارهای فوری و ضروری پیش می‌آید که به User Story ها و Backlog رسمی ربطی ندارند (مثلاً رفع باگ سرور، مشکل زیرساخت). اگر این کارها را در Backlog وارد کنیم، ساختار Sprint به هم می‌ریزد.

راهِحل SmartTask: یک فضای جداگانه «آفرود» برای ثبت این کارها.

پیاده‌سازی فنی:

جزئیات	مقدار
موجودیت	OffroadTask.cs
سرویس	OffroadTaskService.cs
رابط	IOffroadTaskService.cs
کنترلر	OffroadController.cs
View	/Views/Offroad

ویژگی‌ها:

- ثبت کار با عنوان، توضیحات، اولویت و مسئول
- تغییر وضعیت (ToDo → InProgress → Done → Cancelled)
- تغییر اولویت (Low, Normal, High, Critical)
- تخصیص و تغییر مسئول
- حذف نرم (Soft Delete)
- فقط سازنده یا مدیر پروژه اجازه مدیریت دارد

مزیت نسبت به Jira/Trello: در Jira و Trello اگر بخواهید تسک خارج از Sprint ثبت کنید، باید آن را در Backlog وارد کنید. SmartTask با بخش آفرود این مشکل را حل کرده — بدون آسیب به ساختار رسمی.

نواوری ۳: تحلیل بارکاری اعضا (Workload Analysis)

مشکلی که حل می‌کند: مدیر پروژه نمی‌داند هر عضو چقدر بار کاری دارد. ممکن است یک نفر ۱۲۰٪ اضافه‌بار داشته باشد و یک نفر فقط ۳۰٪. بدون این اطلاعات، تسک‌ها ناعادلانه تخصیص داده می‌شوند.

راهِحل SmartTask: محاسبه خودکار درصد اشغال هر عضو بر اساس ساعت تخمینی تسک‌ها و ظرفیت هفتگی.

پیاده‌سازی فنی:

جزئیات	مقدار
سرویس	WorkloadAnalysisService.cs
رابط	IWorkloadAnalysisService.cs
کنترلر	WorkloadController.cs
View	/Views/Workload
ViewModel	WorkloadIndexViewModel , WorkloadMemberViewModel

فرمول محاسبه:

درصد اشغال = (ساعات تخصیص یافته ÷ ظرفیت هفتگی) × ۱۰۰

تفکیک: اگر تسکی به چند نفر تخصیص داده شده باشد:

تسک ÷ تعداد افراد تخصیص یافته = Estimate = ساعات هر فرد

وضعیت‌ها:

- `under` → `80% >` (سبز - زیر ظرفیت)
- `balanced` → `100% - 80%` (آبی - متعادل)
- `overloaded` → `100% <` (قرمز - اضافه بار)

محدوده تحلیل:

- تحلیل کل پروژه: همه تسک‌های باز پروژه
- تحلیل اسپرینت فعال: فقط تسک‌های اسپرینت جاری

بهینه‌سازی:

- از `ComputeAssignmentMap` برای جلوگیری از حلقه‌های $O(n^2)$ استفاده شده
- `Query` تسکی برای دریافت همه تسک‌های باز + فیلتر در حافظه

نواوری ۴: تحلیل تأثیر وابستگی وظایف (Dependency Impact Analysis)

مشکلی که حل می‌کند: وقتی تسک A عقب بیفتد، مدیر پروژه باید دستی بررسی کند کدام تسک‌های دیگر تحت تأثیر قرار می‌گیرند. در پروژه‌های بزرگ با صدها وابستگی، این کار تقریباً غیرممکن است.

راه حل SmartTask: پیمایش خودکار گراف وابستگی و نمایش زنجیره تأثیر.

پیاده‌سازی فنی:

جزئیات	مقدار
سرویس	<code>TaskDependencyService.cs</code>
رابط	<code>ITaskDependencyService.cs</code>
کنترلر	<code>TaskDependencyController.cs</code> , <code>DependencyController.cs</code>
موجودیت	<code>TaskDependency.cs</code>
ViewModel	<code>ImpactedTaskViewModel</code> , <code>DependencyRiskItemViewModel</code>

الگوریتم پیمایش (BFS):

1. تسک مبدأ را پیدا کن
2. تمام تسک‌های وابسته را پیمایش کن BFS با
3. برای هر تسک وابسته، تعداد روزهای تأخیر محاسبه کن

- تاریخ تحویل جدید = تاریخ اصلی + تعداد روزهای تأخیر
- تأخیر را منتقل می‌کنند (IsRequired=true) فقط وابستگی‌های اجباری

محافظت از حلقوی بودن (Cycle Detection):

```
// WouldCreateCycleAsync - معکوس BFS الگوریتم
// برسیم، یعنی حلقوی است taskId شروع کنیم و به dependsOnTaskId اگر از
private async Task<bool> WouldCreateCycleAsync(int taskId, int dependsOnTaskId) {
    // BFS از dependsOnTaskId شروع می‌شود
    // (حلقوی است) true → برسد taskId اگر به
}
```

خروجی‌ها:

- لیست تسک‌های تأثیرپذیر با عمق وابستگی
- تاریخ تحویل اصلی و پیش‌بینی‌شده
- آیا زنجیره اجباری است یا اختیاری

نواوری ۵: تمدید خودکار موعد وظایف وابسته (Overdue Auto-Cascade)

مشکلی که حل می‌کند: وقتی تسک A دیر می‌شود، مدیر پروژه باید دستی موعد تمام تسک‌های وابسته را آپدیت کند. این کار فراموش‌شدنی و زمان‌بر است.

راه‌حل SmartTask: یک سرویس پس‌زمینه هر ۳۰ دقیقه اجرا می‌شود و به صورت خودکار:

پیاده‌سازی فنی:

جزئیات	مقدار
سرویس پس‌زمینه	OverdueCascadeBackgroundService.cs
موجودیت لاگ	OverdueCascadeLog.cs
فاصله اجرا	هر ۳۰ دقیقه
نوع	BackgroundService در ASP.NET Core

نحوه عملکرد:

- هر ۳۰ دقیقه اجرا می‌شود
- را پیدا می‌کند (Cancelled != وضعیت && Done != امروز && وضعیت < DueDate) تسک‌های عقب‌افتاده
- دارد، پردازش می‌شوند `AutoCascadeDependencyDates = true` فقط تسک‌هایی که حداقل یک عضو تخمینیافته
- برای هر تسک عقب‌افتاده، زنجیره وابستگی را بررسی می‌کند
- تأخیر واقعی (نه تخمینی) را به تسک‌های وابسته اجباری منتقل می‌کند
- لاگ ثبت می‌کند (جلوگیری از اعمال تکراری)
- به هر عضو تخمینیافته اعلان می‌دهد
- ثبت می‌کند (Activity Log) فعالیت

مکانیزم جلوگیری از تکرار:

- جدول OverdueCascadeLog ثبت می‌کند چند روز تأخیر اعمال شده
- اگر تأخیر جدید بیشتر از قبلی باشد، فقط مابه‌التفاوت اعمال می‌شود
- اگر تأخیر جدید کمتر یا مساوی باشد، هیچ کاری انجام نمی‌شود

مزیت: مدیر پروژه دیگر نیازی به پیگیری دستی ندارد. وقتی تسک A دیر می‌شود، موعد تسک B (وابسته به A) و تسک C (وابسته به B) هم خودکار عقب کشیده می‌شود.

نواوری ۶: اولویت‌بندی هوشمند وظایف (Smart Priority System)

مشکلی که حل می‌کند: اولویت‌ها معمولاً بر اساس حس مدیر پروژه تعیین می‌شوند. اما واقعاً باید بر اساس فاکتورهای عینی مثل فوریت زمانی، تأثیر وابستگی و بار کاری تعیین شوند.

راه حل SmartTask: الگوریتم امتیازدهی که سه عامل را ترکیب می‌کند.

پیاده‌سازی فنی:

جزئیات	مقدار
سرویس	PriorityEngineService.cs
رابط	IPriorityEngineService.cs
کنترلر	DelayRiskController.cs (بخش Priority)
ViewModel	SmartPriorityViewModel

سه عامل امتیازدهی:

عامل	وزن	منطق
فوریت زمانی	۴۰-۰	اگر موعد گذشته باشد → ۴۰، اگر امروز باشد → ۴۰، هر روز کمتر → کمتر
تأثیر وابستگی	۳۵-۰	هر تسک وابسته اجباری ۷+ امتیاز (حداکثر ۳۵)
ریسک بارکاری	۲۵-۰	اگر مسئول < ۱۰۰٪ اشغال → ۲۵، اگر ۸۰٪-۱۰۰٪ → ۱۵

تبدیل امتیاز به اولویت:

0-20 → Lowest
21-40 → Low
41-60 → Medium
61-80 → High
81-100 → Highest

مثال واقعی:

تسک «تست API» با اولویت «متوسط» ثبت شده. اما: موعدش ۲ روز پیش بوده (فوریت=۴۰)، ۳ تسک بهش وابسته‌اند (وابستگی=۲۱)، مسئولش ۱۱۰٪ اشغال است (بارکاری=۲۵). امتیاز = ۸۶ → پیشنهاد: **Highest**

قابلیت اعمال با یک کلیک: مدیر پروژه می‌تواند پیشنهاد سیستم را با یک کلیک اعمال کند.

نواوری ۷: پیش‌بینی ریسک تأخیر پروژه (Delay Risk Prediction)

مشکلی که حل می‌کند: مدیر پروژه نمی‌داند پروژه‌اش چقدر در معرض تأخیر است. باید تک‌تک گزارش‌ها را بررسی کند.

راه‌حل SmartTask: یک امتیاز ریسک ۰ تا ۱۰۰ با ترکیب ۴ شاخص + تحلیل متنی AI.

پیاده‌سازی فنی:

جزئیات	مقدار
سرویس	DelayRiskService.cs
رابط	IDelayRiskService.cs
کنترلر	DelayRiskController.cs
ViewModel	DelayRiskViewModel

چهار شاخص و وزن آن‌ها:

شاخص	حداکثر امتیاز	منطق محاسبه
نسبت تسک‌های عقب‌افتاده	۴۰	(تعداد عقب‌افتاده ÷ تعداد کل باز) × 40
نسبت اعضای اضافه‌بار	۳۰	(تعداد اضافه‌بار ÷ تعداد کل اعضا) × 30
زنجیره‌های پرریسک وابستگی	۲۰	تعداد زنجیره‌ها × 4 (حداکثر ۲۰)
تمدیدهای خودکار اخیر	۱۰	تعداد تمدیدهای ۷ روز اخیر × 2 (حداکثر ۱۰)

سطوح ریسک:

کم (سبز) → 0-25
متوسط (زرد) → 26-50
بالا (نارنجی) → 51-75
بحرانی (قرمز) → 76-100

تحلیل متنی AI: با کلیک روی دکمه «تحلیل هوشمند»:

- آمار عددی به AI ارسال می‌شود
- AI تحلیل ۳-۴ جمله‌ای به زبان فارسی برمی‌گرداند
- لحن: مثل یک مشاور مدیریت پروژه
- در پایان یک پیشنهاد عملی مشخص

نواوری ۸: شاخص سلامت پروژه (Project Health Score)

مشکلی که حل می‌کند: برای فهمیدن وضعیت کلی پروژه باید وارد چندین صفحه شد.

راه حل SmartTask: یک عدد ترکیبی ۰ تا ۱۰۰ با رنگ سبز تا قرمز.

پیاده‌سازی فنی:

جزئیات	مقدار
سرویس	ProjectHealthService.cs
رابط	IPProjectHealthService.cs
ViewModel	ProjectHealthViewModel

چهار زیرشاخص:

زیرشاخص	وزن	منطق
سلامت زمان‌بندی	۳۰%	100 - (نسبت عقب‌افتاده × 100)
سلامت بارکاری	۲۵%	100 - (نسبت اضافه‌بار × 100)
سلامت وابستگی	۲۰%	100 - (تعداد زنجیره‌ها × 10)
درصد پیشرفت	۲۵%	(تعداد تسک‌های تمام‌شده ÷ کل) × 100

فرمول نهایی:

HealthScore = Schedule × 0.30 + Workload × 0.25 + Dependency × 0.20 + Delivery × 0.25

سطوح و نمادها:

≥ 85 → عالی 😄 (fa-solid fa-face-smile-beam)
≥ 70 → خوب 😊 (fa-solid fa-face-smile)
≥ 50 → نیازمند توجه 😐 (fa-solid fa-face-meh)
< 50 → بحرانی 😞 (fa-solid fa-face-frown)

نواوری ۹: تولید گزارش هوشمند پایان اسپرینت (AI Sprint Report)

مشکلی که حل می‌کند: نوشتن گزارش پایان اسپرینت کاری زمان‌بر و تکراری است.

راه حل SmartTask: AI با تحلیل آمار واقعی اسپرینت، گزارش روایی تولید می‌کند.

پیاده‌سازی فنی:

جزئیات	مقدار
سرویس	SprintReportAiService.cs
رابط	ISprintReportAiService.cs

جزئیات	مقدار
کنترلر	<code>SprintReportController.cs</code>
موجودیت	<code>SprintReport.cs</code>
ViewModel	<code>SprintReportViewModel</code>

داده‌های جمع‌آوری شده:

- نام اسپرینت و پروژه
- هدف اسپرینت
- بازه زمانی
- تکمیل شده (با درصد) vs برنامه‌ریزی شده Story Point
- کل vs تعداد تسک‌های تکمیل شده
- های ناتمام User Story
- بیشترین مشارکت‌کننده‌ها (۳ نفر اول)

پرامپت AI:

System: تو یک اسکرام‌مستر باتجربه هستی...
 Report includes: دستاوردها، نکات ناتمام، پیشنهاد برای اسپرینت بعدی
 Language: فارسی
 Length: تا ۵ جمله ۳

ذخیره‌سازی: گزارش‌ها در جدول `SprintReports` ذخیره می‌شوند و تاریخچه کامل قابل مشاهده است.

نواوری ۱۰: ترید کردن وظایف بین اعضا (Task Trading)

مشکلی که حل می‌کند: وقتی یک عضو درگیر کار دیگری است، باید بتواند تسکش را به همکارش واگذار یا مبادله کند — بدون اینکه بدون اجازه طرف مقابل باشد.

راه حل SmartTask: سیستم درخواست و تأییدیه.

پیاده‌سازی فنی:

جزئیات	مقدار
سرویس	<code>TaskTradeService.cs</code>
رابط	<code>ITaskTradeService.cs</code>
کنترلر	<code>TaskTradeController.cs</code>
موجودیت	<code>TaskTradeRequest.cs</code>
ViewModel	<code>TradeModalDataViewModel</code> , <code>TaskTradeIndexViewModel</code>

انواع ترید:

۱. واگذاری یک‌طرفه: کاربر A تسک خودش را به کاربر B پیشنهاد می‌دهد

2. مبادله دوطرفه: کاربر A تسک X خودش را با تسک Y کاربر B عوض می‌کند

جریان کار:

۱. دکمه «ترید» روی تسک را می‌زند A کاربر
۲. لیست اعضای پروژه نمایش داده می‌شود
۳. را انتخاب می‌کند B کاربر
۴. برای مبادله انتخاب می‌کند B تسکی از کاربر (اختیاری)
۵. پیام اختیاری می‌نویسد
۶. (Status = Pending) درخواست ایجاد می‌شود
۷. اعلان داده می‌شود B به کاربر
۸. قبول یا رد می‌کند B کاربر
۹. ها جابه‌جا می‌شوند Assignment → اگر قبول
۱۰. اگر رد → درخواست لغو می‌شود

قوانین امنیتی:

- نمی‌توان با خودتان ترید کنید
- تسک باید به شما تخصیص داده شده باشد
- نمی‌توان درخواست تکراری Pending برای یک تسک ایجاد کرد
- فقط دریافت‌کننده حق پاسخ دارد
- فقط فرستنده حق لغو درخواست را دارد

نواوری ۱۱: گراف وابستگی وظایف (Dependency Graph)

مشکلی که حل می‌کند: خواندن لیست‌های متنی وابستگی‌ها دشوار است. مدیر پروژه باید کل نقشه وابستگی را با یک نگاه ببیند.

راه حل SmartTask: نمای گرافیکی تسک‌ها به صورت گره و یال.

پیاده‌سازی فنی:

جزئیات	مقدار
سرویس	TaskDependencyService.cs
رابط	ITaskDependencyService.cs
کنترلر	DependencyController.cs
متد اصلی	GetDependencyGraphAsync()
ViewModel	DependencyGraphViewModel , DependencyGraphNodeViewModel , DependencyGraphEdgeViewModel
View	Views/Dependency/_DependencyPartial.cshtml
کتابخانه فرانت	Cytoscape.js + Dagre Layout (لوکال)
CSS	wwwroot/css/dependency.css

جزئیات	مقدار
JS	wwwroot/js/dependency-graph.js

ساختار خروجی:

```
DependencyGraphViewModel {
  Nodes: List<DependencyGraphNodeViewModel> {
    Id, Title, IsDone, IsOverdue, IsAtRisk
  },
  Edges: List<DependencyGraphEdgeViewModel> {
    SourceTaskId, TargetTaskId, IsRequired
  }
}
```

رنگ‌بندی گره‌ها:

- سبز: تسک تمام‌شده
- قرمز: تسک عقب‌افتاده
- نارنجی: تسک در معرض ریسک
- خاکستری: تسک عادی

نحوه عملکرد:

1. کاربر وارد صفحه پروژه می‌شود و تب «وابستگی» را انتخاب می‌کند
2. سپس تب «نمای گراف» را می‌زند
3. `TaskDependencyService.GetDependencyGraphAsync()` فراخوانی می‌شود
4. گره‌ها (تسک‌ها) و یال‌ها (وابستگی‌ها) از دیتابیس استخراج می‌شوند
5. داده‌ها به صورت JSON به مرورگر ارسال می‌شوند
6. Cytoscape.js با Layout Dagre گراف را رندر می‌کند
7. کاربر می‌تواند روی گره‌ها کلیک کند و جزئیات تسک را ببیند

ویژگی‌های فنی:

- **Fallback Layout**: اگر Dagre fail بشه، Grid Layout استفاده می‌شود
- **waitForDimensions**: بررسی ابعاد کانتینر قبل از رندر (جلوگیری از صفحه خالی)
- **cy.fit()**: تنظیم خودکار viewport روی نودها
- **CDN لوکال**: کتابخانه‌ها از فایل لوکال سرو می‌شوند (سازگاری با فیلترشکن)

مزیت نسبت به نمای لیستی:

در نمای لیستی، مدیر باید تک‌تک وابستگی‌ها را بخواند. در نمای گرافیکی، کل نقشه وابستگی با یک نگاه قابل فهم است و تسک‌های پرریسک (عقب‌افتاده) فوراً مشخص می‌شوند.

نواوری ۱۲: معماری تصمیم‌یار مبتنی بر LLM (Decision Support Architecture)

مشکلی که حل می‌کند: اکثر سیستم‌های مدیریت پروژه فقط اطلاعات را ذخیره و نمایش می‌دهند. حتی اگر هوش مصنوعی هم داشته باشند، فقط متن تولید می‌کنند و ربطی به داده‌های واقعی پروژه ندارد.

راه حل SmartTask: یک معماری تصمیم‌یار که الگوریتم و LLM را ترکیب می‌کند. LLM فقط متن نمی‌نویسد – خروجی JSON ساختاریافته برمی‌گرداند که به شاخص‌های عددی پروژه تبدیل می‌شود.

پیاده‌سازی فنی:

جزئیات	مقدار
سرویس AI	<code>AiClientService.cs</code> – متد <code><GetStructuredCompletionAsync>T</code>
مدل‌های خروجی	<code>AiRiskAnalysisResult</code> , <code>AiPriorityReasonResult</code> , <code>AiHealthAnalysisResult</code>
لاگ تصمیمات	<code>AiDecisionLogService.cs</code> + entity <code>AiDecisionLog</code>
سرویس‌های ترکیبی	<code>GetRiskOverviewWithAiAsync</code> , <code>GetSuggestionWithAiAsync</code> , <code>GetHealthWithAiAsync</code>

چرخه کامل:

خص عددی → ۵ پیشنهاد → ۶ قبول/رد مدیر → ۷ ثبت تصمیم → ۴ LLM (JSON) → ۳ دیتابیس → ۲ تحلیل الگوریتمی → ۱

نمونه خروجی LLM (JSON):

```
{
  "risk_score": 75,
  "risk_level": "high",
  "factors": ["تأخیر 3 تسک", "اضافه بار 2 عضو"],
  "suggestion": "اول تسک‌های عقب افتاده رو حل کنید",
  "confidence": "high",
  "summary": "وضعیت ریسک بالاست"
}
```

ثبت تصمیمات:

- هر بار که مدیر پیشنهاد AI رو قبول/رد کنه، در `AiDecisionLogs` ثبت می‌شود
- فیلدها: نوع تصمیم، امتیاز AI، پیشنهاد متنی، دلایل، تصمیم مدیر (پذیرش/رد)
- این اطلاعات برای ارزیابی صحت AI در پایان‌نامه استفاده می‌شود

۷. تکنولوژی‌های استفاده شده

۷.۱ هوش مصنوعی (OpenAI-compatible API)

چپست؟ SmartTask از یک API سازگار با OpenAI استفاده می‌کند. مدل اصلی Qwen3-4B است که روی سرور LM Studio اجرا می‌شود.

نحوه اتصال:

```
// Services/AI/AIClientService.cs
// آدرس: http://92.246.145.99:1234/v1/chat/completions
// مدل: qwen/qwen3-4b
// روش: HTTP POST با JSON body
// max_tokens محدودیت: ۶۰ ثانیه تایم اوت، ۴۰۰
```

فایل‌های مرتبط:

- `Services/AI/IAIClientService.cs` – رابط
- `Services/AI/AIClientService.cs` – پیاده‌سازی
- `Services/AI/OpenAiSettings.cs` – تنظیمات (ApiKey, Model, BaseUrl)

کجاها استفاده می‌شود:

1. `TaskBreakdownService` – تجزیه تسک (خروجی: JSON array)
2. `DelayRiskService` – تحلیل ساختاریافته ریسک (خروجی: JSON با risk_score, factors, suggestion)
3. `PriorityEngineService` – دلایل تکمیلی اولویت (خروجی: JSON با ai_reasons, explanation)
4. `ProjectHealthService` – تحلیل جامع سلامت پروژه (خروجی: JSON با critical_areas, recommendations)
5. `SprintReportAiService` – گزارش اسپرینت (خروجی: متن فارسی)

نحوه دریافت خروجی ساختاریافته:

```
// Services/AI/AIClientService.cs
// متد GetStructuredCompletionAsync<T>:
// 1. System Prompt تقویت شده (الزام JSON)
// 2. دریافت پاسخ از LLM
// 3. اگر وجود داشته باشد Markdown code fence حذف
// 4. JsonSerializer به کلاس مورد نظر با Deserialize
// 5. (Fallback) برگردان default(T) → نبود JSON اگر
```

پرامپت‌ها (به زبان فارسی):

- همه پرامپت‌ها به فارسی نوشته شده‌اند
- از `no_think/` برای جلوگیری از تفکر اضافی مدل استفاده شده
- temperature بین ۰.۵ تا ۰.۷ (برای تعادل بین خلاقیت و دقت)

SignalR ۷.۲ (ارتباط بلادرنگ)

چیست؟ SignalR یک کتابخانه میکروسافت برای ارتباط Real-time بین سرور و کلاینت است. از WebSocket استفاده می‌کند (با fallback به Long Polling).

کجاها استفاده شده:

هاب	آدرس	کاربرد
NotificationHub	hubs/notification/	اعلان‌های فوری

ه‌اب	آدرس	کاربرد
ChatHub	hubs/chat/	چت گروهی پروژه

:NotificationHub

- کاربر با اتصال، به گروه `user-{userId}` اضافه می‌شود
- سرور اعلان‌ها را فقط به همان گروه ارسال می‌کند
- `NotificationService` از `hubContext.Clients.Group_ (...)` استفاده می‌کند

:ChatHub

- کاربر به گروه‌های `project-chat-{projectId}` اضافه می‌شود
- قابلیت‌ها: ارسال پیام، ویرایش، حذف، تایپ، ری‌اکشن، سنجاق
- وضعیت آنلاین/آفلاین با `PresenceTracker` (Singleton)
- `Webpushr` با `Push Notification`

Email Service ۷.۳

چیس‌ت؟ ارسال ایمیل از طریق SMTP.

فایل‌ها:

- تنظیمات (Host, Port, Email, Password, EnableSsl) — `Services/Email/EmailSettings.cs`
- رابط — `Services/Email/IEmailService.cs`
- پیاده‌سازی با `System.Net.Mail.SmtpClient` — `Services/Email/EmailService.cs`

کجا‌ها استفاده می‌شود:

- ارسال ایمیل دعوت‌نامه `Workspace` — `WorkspaceInvitationService`
- ایمیل شامل لینک ثبت‌نام یا پذیرش دعوت
- قالب HTML با دکمه استایل‌دار

Webpushr (Push Notification) ۷.۴

چیس‌ت؟ `Webpushr` یک سرویس ارسال `Push Notification` وب است. وقتی کاربر در مرورگر اشتراک داشته باشد، حتی اگر صفحه باز نباشد، اعلان دریافت می‌کند.

فایل: `Services/Implementations/WebpushrService.cs`

نحوه کار:

1. کاربر در مرورگر اشتراک `Push` می‌کند
2. شناسه `WebpushrSubscriberId` در `ApplicationUser` ذخیره می‌شود
3. هنگام ارسال پیام چت، به همه اعضای آفلاین `Push` ارسال می‌شود
4. API: `POST https://api.webpushr.com/v1/notification/send/sid`

۷.۵ Background Services (سرویس‌های پس‌زمینه)

چیست؟ در `BackgroundService`، ASP.NET Core کلاسی است که کدی را در پس‌زمینه و به صورت دوره‌ای اجرا می‌کند.

سرویس‌های پس‌زمینه `SmartTask`:

سرویس	فایل
<code>OverdueCascadeBackgroundService</code>	<code>Infrastructure/BackgroundJobs/OverdueCascadeBackgroundService.cs</code>
<code>ReminderBackgroundService</code>	<code>Infrastructure/BackgroundJobs/ReminderBackgroundService.cs</code>

ثبت در `Program.cs`:

```
builder.Services.AddHostedService<ReminderBackgroundService>();
builder.Services.AddHostedService<OverdueCascadeBackgroundService>();
```

۷.۶ QuestPDF و ClosedXML

`QuestPDF`: تولید فایل‌های PDF از گزارش‌ها

- لایسنس Community (رایگان)
- ثبت: `QuestPDF.Settings.License = QuestPDF.Infrastructure.LicenseType.Community`;

`ClosedXML`: تولید فایل‌های Excel از گزارش‌ها

۸. فایل ساختار پروژه

```
SmartTask.Web/
├── Program.cs           # نقطه ورود برنامه
├── SmartTask.Web.csproj # پیکربندی پروژه
├── appsettings.json     # تنظیمات (connection string, AI, Email, Webpushr)
├──
├── Common/
│   ├── Attributes/    # Attribute های سفارشی
│   ├── Extensions/    # Extension Methods
│   ├── Filters/        # Action Filters (مثل CurrentContextFilter)
│   └── Helpers/        # کلاس‌های کمکی
├──
├── Controllers/        # MVC کنترلر 41
├── AccountController.cs # احراز هویت
```

```
| ProjectController.cs # مدیریت پروژه
| SprintController.cs # مدیریت اسپرینت
| TaskController.cs # مدیریت تسک
| TaskDependencyController.cs # وابستگی‌ها
| OffroadController.cs # آفرود
| WorkloadController.cs # بارکاری
| DelayRiskController.cs # ریسک و اولویت هوشمند
| TaskTradeController.cs # ترید تسک
| SprintReportController.cs # گزارش اسپرینت
| ChatController.cs # چت
| ... (کنترلر دیگر 30+)

| Data/
| | Context/
| | | ApplicationDbContext.cs # DbContext اصلی
| | Configurations/ # 35 Fluent API Configuration (شامل AiDecisionLogConfiguratio
| Hubs/
| | NotificationHub.cs # هاب اعلان‌ها (SignalR)
| | ChatHub.cs # هاب چت (SignalR)

| Infrastructure/
| | BackgroundJobs/
| | | OverdueCascadeBackgroundService.cs # تمديد خودكار
| | | ReminderBackgroundService.cs # يادآوری‌ها
| | Interfaces/ # IGenericRepository, IUnitOfWork
| | Repositories/
| | | GenericRepository.cs # Repository عمومی
| | | UnitOfWork.cs # الگوی Unit of Work
| | Seed/
| | | RoleSeeder.cs # ایجاد نقش‌ها
| | | AdminSeeder.cs # ایجاد ادمین پیشفرض
| | Services/
| | | CurrentUserService.cs # کاربر فعلی
| | | CurrentContextService.cs # Context فعلی
| | | PresenceTracker.cs # ردیابی آنلاین/آفلاین

| Migrations/ # EF Core مایگريشن‌های

| Models/
| | Entities/ # 35 (شامل AiDecisionLog) موجودیت داده‌ای
| | Enums/ # 26 (شامل AiDecisionType, AiUserDecision, AiDecisionEnt
| | ViewModels/ # 30+ (شامل Ai/ با 3 مدل خروجی) ViewModel پوشه
| | DTOs/ # BurndownPointDto, VelocityPointDto
| | Navigation/ # مدل‌های ناوبری

| Services/
| | AI/
| | | IAIClientService.cs # رابط AI (شامل GetStructuredCompletionAsync<T>)
| | | AiClientService.cs # پیاده‌سازی AI (HTTP + Deserialize JSON)
| | | OpenAiSettings.cs # تنظیمات AI
| | Email/
| | | IEmailService.cs # رابط ایمیل
| | | EmailService.cs # پیاده‌سازی SMTP
| | | EmailSettings.cs # تنظیمات SMTP
| | Files/
| | | FileUploadService.cs # آپلود فایل
| | Interfaces/ # رابط سرویس 45
| | Implementations/ # پیاده‌سازی سرویس 45
```

```

└─ TaskBreakdownService.cs      # تجزیه هوشمند
└─ WorkloadAnalysisService.cs   # تحلیل بارکاری
└─ TaskDependencyService.cs     # وابستگی و تحلیل تأثیر
└─ PriorityEngineService.cs     # اولویتبندی هوشمند
└─ DelayRiskService.cs         # ریسک تأخیر
└─ ProjectHealthService.cs     # سلامت پروژه
└─ SprintReportAiService.cs     # اسپرینت AI گزارش
└─ OffroadTaskService.cs       # آفرود
└─ TaskTradeService.cs         # ترید تسک
└─ AiDecisionLogService.cs     # AI لاگ تصمیمات
└─ NotificationService.cs      # اعلان‌ها
└─ ChatService.cs              # چت
└─ ... (سرویس دیگر 35+)

└─ Views/                      # View های Razor
    └─ Shared/                  # View های مشترک
    └─ Home/                    # صفحه اصلی
    └─ Account/                 # ورود/ثبت‌نام
    └─ Project/                 # پروژه
    └─ Sprint/                  # اسپرینت
    └─ Task/                    # تسک
    └─ Workload/                # بارکاری
    └─ Offroad/                 # آفرود
    └─ DelayRisk/               # ریسک
    └─ TaskTrade/               # ترید
    └─ Dependency/              # وابستگی
    └─ Chat/                    # چت
    └─ ... (پوشه 30+)

└─ wwwroot/                    # فایل‌های استاتیک
    └─ css/                     # استایل‌ها
    └─ js/                      # اسکریپت‌ها
    └─ lib/                     # کتابخانه‌های فرانت‌اند
    └─ fonts/Vazirmatn/         # فونت فارسی وزیرمتن
    └─ uploads/                 # فایل‌های آپلود شده

└─ SmartTask.Web.Tests/        # تست‌های واحد
    └─ Services/
        └─ TaskDependencyServiceTests.cs
        └─ WorkloadAnalysisServiceTests.cs
        └─ PriorityEngineServiceTests.cs
        └─ ProjectHealthServiceTests.cs
        └─ SprintServiceTests.cs
        └─ SubTaskServiceTests.cs
        └─ TaskServiceTests.cs
        └─ DateFormatServiceTests.cs
    └─ TestHelpers/
        └─ TestDataBuilder.cs
        └─ TestDbContextFactory.cs

```

۹. تست‌ها و کیفیت کد

تست‌های واحد (Unit Tests)

فایل‌های تست:

تست	سرویس تست‌شده	تمرکز
TaskDependencyServiceTests	TaskDependencyService	چرخه وابستگی، BFS، محاسبه تأخیر
WorkloadAnalysisServiceTests	WorkloadAnalysisService	محاسبه بارکاری
PriorityEngineServiceTests	PriorityEngineService	امتیازدهی هوشمند
ProjectHealthServiceTests	ProjectHealthService	شاخص سلامت
SprintServiceTests	SprintService	مدیریت اسپرینت
SubTaskServiceTests	SubTaskService	زیروظایف
TaskServiceTests	TaskService	CRUD تسک
DateFormatServiceTests	DateFormatService	تاریخ شمسی/میلادی

Helper ها:

- TestDataBuilder – ساخت داده‌های تستی
- TestDbContextFactory – ایجاد DbContext با InMemory Database

نکات بهینه‌سازی کد

- Soft Delete با ViewState در BaseEntity
- ExecuteUpdateAsync به جای Load-Modify-Save (عملکرد بهتر)
- Batch Operations برای اعلان‌ها و فعالیت‌ها
- In-Memory Traversal برای گراف وابستگی (جلوگیری از N+1 Query)
- Pre-computed Maps برای محاسبه بارکاری

۱۰. نقشه راه و وضعیت پیاده‌سازی

وضعیت	فیچر
تکمیل ✓	Workspace Management
تکمیل ✓	Project Management
تکمیل ✓	Sprint Management
تکمیل ✓	Backlog & User Stories
تکمیل ✓	Task Management

وضعیت	فیچر
تکمیل 	AI Task Breakdown
تکمیل 	Offroad Tasks
تکمیل 	Workload Analysis
تکمیل 	Dependency Impact Analysis
تکمیل 	Overdue Auto-Cascade
تکمیل 	Smart Priority System
تکمیل 	Delay Risk Prediction
تکمیل 	Project Health Score
تکمیل 	AI Sprint Report
تکمیل 	Task Trading
تکمیل 	Decision Support Layer (LLM)
تکمیل 	Dependency Graph (بصری)
تکمیل 	Real-time Chat
تکمیل 	Notifications
تکمیل 	Push Notifications (Webpushr)
تکمیل 	Email Service
تکمیل 	File Upload
تکمیل 	Reports (PDF/Excel)
تکمیل 	Unit Tests

پرسش‌های متداول احتمالی اساتید

س: چرا از هوش مصنوعی استفاده کردید؟

ج: هوش مصنوعی در سه بخش کلیدی استفاده شده: (۱) تجزیه خودکار تسک‌ها برای کمک به اعضای کم‌تجربه، (۲) تحلیل متنی ریسک پروژه به جای اعداد خام، (۳) تولید خودکار گزارش اسپرینت. در هر سه مورد، هدف کاهش بار کاری مدیر پروژه و افزایش سرعت تصمیم‌گیری بوده.

س: تفاوت SmartTask با Jira یا Trello چیست؟

ج: (۱) بخش آف‌رود برای کارهای خارج از دامنه، (۲) تحلیل بارکاری خودکار، (۳) تمدید خودکار وابسته‌ها، (۴) اولویت‌بندی هوشمند با الگوریتم، (۵) گزارش‌گیری (۶) AI، تجزیه تسک با (۷) AI، ترید تسک با سیستم درخواست و تأییدیه.

س: چرا SignalR استفاده کردید؟

ج: برای دو کاربرد اصلی: (۱) اعلان‌های فوری – وقتی تسک جدیدی تخصیص داده شود، کاربر فوراً اعلان بگیرد، (۲) چت گروهی – پیام‌رسانی Real-time بدون رفرش صفحه.

س: مکانیزم جلوگیری از حلقوی شدن وابستگی‌ها چیست؟

ج: الگوریتم BFS معکوس. وقتی کاربر می‌خواهد تسک B را وابسته به A کند، سیستم از B شروع به پیمایش می‌کند. اگر به A برسد، یعنی حلقوی است و خطا نشان می‌دهد.

س: تمدید خودکار چگونه از تکرار جلوگیری می‌کند؟

ج: جدول OverdueCascadeLog ثبت می‌کند چند روز از منبع به مقصد اعمال شده. اگر تأخیر جدید بیشتر از قبلی باشد، فقط مابه‌التفاوت اعمال می‌شود. اگر کمتر یا مساوی باشد، کاری انجام نمی‌شود.

س: هوش مصنوعی چگونه متصل می‌شود؟

ج: از طریق HTTP API سازگار با OpenAI. مدل Qwen3-4B روی سرور LM Studio اجرا می‌شود. درخواست‌ها با POST ارسال و پاسخ JSON دریافت می‌شود. تایم‌اوت ۶۰ ثانیه و حداکثر ۴۰۰ توکن تنظیم شده.

س: Smart Task چه مدل‌های داده‌ای دارد؟

ج: سلسله‌مراتبی: TaskDependency بین Workspace → Project → Backlog → UserStory → TaskItem → SubTaskItem تسک‌ها، TaskAssignment برای تخصیص، و بسیاری موجودیت‌های کمکی مثل Notification, Comment, Attachment, Reminder, ActivityLog, TimeLog.

س: آیا فقط هوش مصنوعی وصل کردید یا معماری تصمیم‌یار دارید؟

ج: SmartTask یک معماری تصمیم‌یار مبتنی بر LLM دارد. تفاوت این است که: (۱) داده‌های ساختاریافته پروژه از دیتابیس استخراج و تحلیل الگوریتمی می‌شوند، (۲) داده‌ها به LLM ارسال و خروجی JSON ساختاریافته دریافت می‌شود (نه متن آزاد)، (۳) خروجی AI به شاخص‌های عددی پروژه تبدیل می‌شود، (۴) مدیر می‌تواند پیشنهاد را قبول/رد کند، (۵) تصمیم مدیر ثبت می‌شود تا صحت AI قابل ارزیابی باشد.

س: تصمیمات هوش مصنوعی چگونه ارزیابی می‌شوند؟

ج: جدول AiDecisionLogs تمام تصمیمات مدیر (قبول/رد) را ثبت می‌کند. فیلدها شامل نوع تصمیم، امتیاز AI، پیشنهاد، دلایل و تصمیم نهایی مدیر است. این اطلاعات برای محاسبه نرخ پذیرش AI و بهبود پرامپت‌ها استفاده می‌شود.